

Task 3: English + Devanagari OCR & Text Preparation

Indian-Context-Aware Multilingual and Multimodal Web Retrieval & Summarization

Team members: PRAVEEN (230041018) | VIVEK (230041014)

1. Sub-Problem

Convert heterogeneous, on-the-fly collected web documents (HTML pages, text-based PDFs, scanned PDFs, and images, including photographed text such as temple or museum boards) into clean, machine-readable text passages in English and Devanagari/Hindi/Sanskrit. The module must detect whether a document already contains extractable text or needs OCR, apply the correct extraction path, and produce normalized text with full provenance (source, page/image, script, language) and an OCR confidence score, so Task 4 and Task 5 can trust and trace every passage.

2. Input & Expected Output

Input: Raw HTML, PDF, and image documents collected by Task 2, along with basic source metadata (URL, title, domain, content type).

Output: Clean text passages, each tagged with script (Latin, Devanagari, or mixed), language (English, Hindi, Sanskrit, or other), extraction method (direct text vs OCR), OCR confidence score, and a source/page reference linking back to the original document.

API contracts:

extract_and_prepare(document) → passages_with_provenance_and_confidence

detect_script(text) → script_label (Latin / Devanagari / Mixed)

detect_language(text) → language_label (English / Hindi / Sanskrit / Other)

3. Proposed Workflow / Methodology

- Detection and routing: check each PDF/HTML for an existing extractable text layer (e.g. via PyMuPDF/pdfplumber); route text-bearing documents to direct extraction and image-only/scanned documents to OCR.
- OCR engine evaluation: benchmark freely available transformer-based OCR models for English and Devanagari against a classical baseline (e.g. Tesseract) to select the best practical option per script.
- Multimodal handling: support printed Devanagari, scanned Hindi/Sanskrit book pages, English scans, mixed-script pages, and text embedded within photographs.
- Script detection (separate function): identify whether extracted text is Latin, Devanagari, or mixed script, using Unicode code-point range analysis on the extracted characters.
- Language detection (separate function): identify whether the content is English, Hindi, Sanskrit, or another language, independent of script. Since Hindi and Sanskrit share Devanagari, this uses statistical/lexical language-ID models rather than script alone.
- Provenance tagging: attach document ID, page/image reference, detected script, and detected language to every extracted passage.
- Confidence scoring: capture per-passage OCR confidence where available; flag low-confidence text explicitly rather than passing it through silently.
- Text normalization: apply Unicode normalization (important for Devanagari combining characters), strip OCR artefacts, repair broken line/paragraph breaks, and standardize formatting.
- Content-level deduplication: identify and remove identical or near-duplicate articles, paragraphs, or OCR-extracted passages.

4. Existing Tools, Models & Papers to Study

- Tesseract OCR, used as a non-transformer baseline for both English and Devanagari.
- Transformer-based OCR models such as TrOCR and Indic-script variants / IndicOCR-style projects, for improved accuracy.
- PyMuPDF and pdfplumber, for detecting and extracting existing text layers from PDFs.
- Unicode block/range references for script detection; language-ID libraries (langdetect, fastText, IndicLID) for English/Hindi/Sanskrit.
- Unicode normalization references for Devanagari (NFC normalization, combining character handling).

5. Data / Test Cases for Evaluation

- A small curated set of English and Devanagari images/scans with verified ground-truth transcriptions, plus real text-based/scanned PDFs and photographed text sampled from Task 2 outputs.
- A mixed-script test page (English and Devanagari together), and Devanagari passages containing both Hindi and Sanskrit, to validate script vs language detection.
- A set of near-duplicate passages (e.g. the same article copied across multiple sites) to validate content-level deduplication.

6. Evaluation Metrics & Baseline

- Character Error Rate (CER) and Word Error Rate (WER) against ground-truth transcriptions.
- Script-wise performance reported separately for English vs Devanagari.
- Script detection accuracy and language detection accuracy, evaluated separately against labelled test passages.
- OCR confidence calibration, checking whether stated confidence scores track actual accuracy.
- Baseline: Tesseract OCR output, compared against the selected transformer-based model(s).

7. Expected First-Phase Deliverable & Downstream Connection

A working `extract_and_prepare()` module, along with separate `detect_script()` and `detect_language()` functions, returning clean, normalized text passages with script, language, extraction method, OCR confidence, provenance, and content-level deduplication. This feeds Task 4 (relevance ranking) and Task 5 (reliability assessment).